Android Lab: Set 0 and Set 2

Theory and Execution Experiment · Java · Android Studio

Set 0

> **1. Theory:** Explain TextView, EditText and Button in Android. How can font size, font style and text color be set?

TextView

- A **TextView** (`android.widget.TextView`) is a GUI component that **displays text** to the user.
- The user can only read the text, not edit it. It is used for labels, headings and messages.
- Common attributes: `android:text`, `android:textSize`, `android:textColor`, `android:textStyle`, `android:gravity`.
- In Java: `setText("Hello")` changes the text.

EditText

- An **EditText** is a subclass of TextView that lets the user **type and edit text**, for example a name, password or number.
- Common attributes: `android:hint` (grey placeholder text) and `android:inputType` (`text`, `number`, `textPassword`, `phone`, `textEmailAddress`).
- In Java: `String s = editText.getText().toString();` reads what the user typed.

Button

- A **Button** is a subclass of TextView that the user **taps to perform an action**.
- The tap is handled with `setOnClickListener()` in Java, or with `android:onClick` in XML.

Example (XML) (layout_width and layout_height left out to keep it short)

```
<TextView
  android:text="Hello"
  android:textSize="20sp"
  android:textStyle="bold"
  android:textColor="#FF0000" />
```

```
<EditText
  android:hint="Enter your name"
  android:inputType="text" />
```

```
<Button
  android:text="Submit" />
```

Setting font size, font style and text color

Property	In XML	In Java
Font size	`android:textSize="20sp"`	`textView.setTextSize(20);`
Font style	`android:textStyle="bold"` (normal, bold, italic, bold|italic) `android:typeface="serif"` (sans, serif, monospace)	`textView.setTypeface(Typeface.SERIF, Typeface.BOLD);`
Text color	`android:textColor="#FF0000"`	`textView.setTextColor(Color.RED);`
Background color	`android:background="#FFFF00"`	`view.setBackgroundColor(Color.YELLOW);`

sp (scale-independent pixels) is used for text size, so the text scales with the user's font settings. Colors are written as **#RRGGBB** hex values or with constants like `Color.RED`.

Set 0

> **2. Execution Experiment:** To develop a Simple Android/iOS Application that uses GUI components, Font and Colors.

STEP 1 Create the project

1. Open Android Studio and click **New Project**.
2. Select **Empty Views Activity**, then click **Next**.
3. **Name:** `Set 0` · **Package name:** `com.example.set0` (fills in automatically) · **Language:** `Java` · **Minimum SDK:** leave the default
4. Click **Finish** and wait until the loading bar at the bottom stops.

STEP 2 Paste the layout (XML)

> **Where:** left panel → **app** → **res** → **layout** → **activity_main.xml**. If you see a phone picture instead of code, click **Code** at the top right.
> **Do:** Ctrl + A (select all), then Ctrl + V (paste). On Mac use Cmd.

`activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <TextView
        android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World"
        android:textSize="20sp" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />

</LinearLayout>
```

STEP 3 Paste the Java code

> **Where:** left panel → **app** → **java** (may say **kotlin+java**) → **com.example.set0** → **MainActivity**
> **Do:** Ctrl + A, then Ctrl + V.

MainActivity.java

```
package com.example.set0;

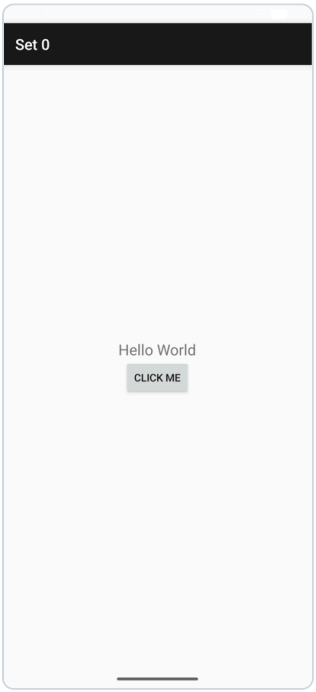
import android.graphics.Color;
import android.graphics.Typeface;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);

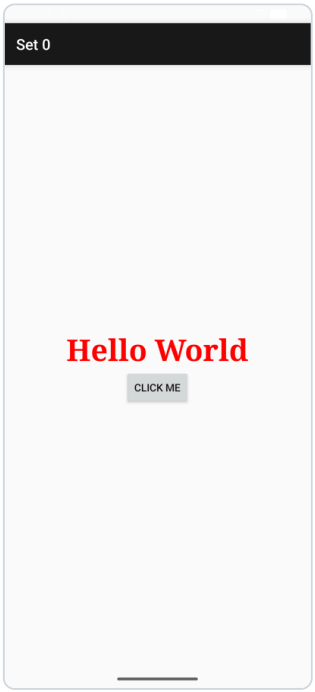
        TextView t = findViewById(R.id.text);
        Button btn = findViewById(R.id.button);

        btn.setOnClickListener(v -> {
            t.setTextSize(40);                          // font size
            t.setTypeface(Typeface.SERIF, Typeface.BOLD); // font style
            t.setTextColor(Color.RED);                  // color
        });
    }
}
```

STEP 4 Run and output



Before tapping



After tapping CLICK ME

Click the green ▶ **Run** button (full steps on the last page).

Output: The screen shows "Hello World" and a **CLICK ME** button. When the button is tapped, the text becomes **size 40**, **bold serif** and **red**.

How it works: The XML places a TextView and a Button. `findViewById` links them to Java, and `setOnClickListener` changes the font size, font style and color when the button is tapped.

Result: A simple Android application using GUI components, font and colors was developed and executed successfully.

Set 2

> **1. Theory:** What are graphical primitives? Explain how Canvas and Paint are used to draw shapes in Android.

Graphical primitives

Graphical primitives are the basic shapes that all graphics are built from. Complex drawings are combinations of them.

- **Point**: a single dot at (x, y)
- **Line**: joins two points
- **Rectangle**: four sides, given by left, top, right, bottom
- **Circle**: given by a center (cx, cy) and a radius
- **Oval / Ellipse** and **Arc**: drawn inside a rectangle
- **Text**: a string drawn at a position

Canvas

- **Canvas** (`android.graphics.Canvas`) is the **drawing surface**. It provides methods that draw each primitive.
- Coordinates are in pixels. **(0, 0) is the top-left corner**; x increases to the right and y increases downward.

Canvas method	Draws
`drawPoint(x, y, paint)`	a point
`drawLine(x1, y1, x2, y2, paint)`	a line between two points
`drawRect(left, top, right, bottom, paint)`	a rectangle
`drawCircle(cx, cy, radius, paint)`	a circle
`drawOval(left, top, right, bottom, paint)`	an oval inside that rectangle
`drawText(text, x, y, paint)`	text
`drawColor(color)`	fills the whole background

Paint

- **Paint** (`android.graphics.Paint`) describes **how** to draw: color, thickness, fill and text size.

Paint method	Purpose
`setColor(Color.RED)`	drawing color
`setStrokeWidth(10)`	thickness of lines and outlines
`setStyle(Paint.Style.FILL / STROKE)`	filled shape or outline only
`setTextSize(60)`	size of text drawn with drawText
`setAntiAlias(true)`	smooth edges

Steps to draw shapes

1. Create a class that **extends View**.
2. Override `onDraw(Canvas canvas)`. Android calls it automatically whenever the view is shown.
3. Create a **Paint** object and set its color, stroke width and text size.
4. Call the Canvas draw methods, for example `canvas.drawCircle(700, 450, 150, paint)`.
5. Show the view with `setContentView(new MyView(this))`. Calling `invalidate()` redraws it.

Set 2

> **2. Execution Experiment:** To develop a Simple Android/iOS Application that draws basic Graphical Primitives on the screen.

STEP 1 Create the project

1. **File** → **New** → **New Project** (or **New Project** on the welcome screen).
2. Select **Empty Views Activity**, then click **Next**.
3. **Name:** `Set 2` · **Package name:** `com.example.set2` · **Language:** `Java`
4. Click **Finish** and wait for loading to stop.

STEP 2 Paste the Java code (no XML change needed)

> **Where:** left panel → **app** → **java** (may say **kotlin+java**) → **com.example.set2** → **MainActivity**
> **Do:** Ctrl + A, then Ctrl + V. On Mac use Cmd.

MainActivity.java

```
package com.example.set2;

import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.os.Bundle;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(new View(this) {
            protected void onDraw(Canvas c) {
                Paint p = new Paint();
                p.setStrokeWidth(10);
                p.setTextSize(60);

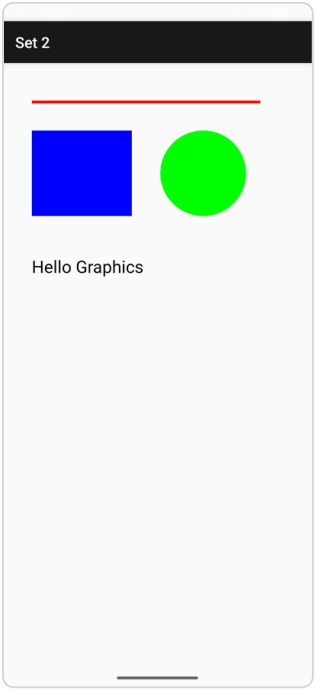
                p.setColor(Color.RED);
                c.drawLine(100, 200, 900, 200, p);         // line

                p.setColor(Color.BLUE);
                c.drawRect(100, 300, 450, 600, p);         // rectangle

                p.setColor(Color.GREEN);
                c.drawCircle(700, 450, 150, p);            // circle

                p.setColor(Color.BLACK);
                c.drawText("Hello Graphics", 100, 800, p); // text
            }
        });
    }
}
```

STEP 3 Run and output



Output

Click the green ▶ **Run** button (full steps in How to Run below).

Output: a red **line**, a blue **rectangle**, a green **circle** and the **text** "Hello Graphics".

How it works: Instead of an XML layout, the app shows a custom View. Android calls its `onDraw()` with a Canvas. Paint sets the color, thickness and text size, and `drawLine`, `drawRect`, `drawCircle` and `drawText` draw the primitives at the given (x, y) pixel positions.

Result: A simple Android application that draws basic graphical primitives was developed and executed successfully.

How to Run (same for Set 0 and Set 2)

1. Look at the top toolbar for the **device dropdown**, next to the green ▶ **Run** button.
2. **If a phone name is shown** (for example *Medium Phone*), go to step 4.
3. **If it says "No Devices":**
   a. Open **Device Manager**: click the phone icon on the right side, or use **View** → **Tool Windows** → **Device Manager**.
   b. Click **+**, then **Create Virtual Device**, choose **Medium Phone** (or any phone), and click **Next** → **Finish**.
   c. Click ▶ next to the new phone to start it.
4. Click the green ▶ **Run** button, or press Shift + F10 on Windows or Ctrl + R on Mac.
5. Wait for the emulator to start. Your app opens on it automatically.

If something goes wrong

Problem	Fix
`R` or the code is red	**Build** → **Rebuild Project** and wait.
Error on the first line `package ...`	You gave the project a different name. Change the first line to the `package` line Android Studio made, for example `package com.example.myapplication;`
Can't find the **java** folder	Set the dropdown at the top of the left panel to **Android**.
Layout file shows a phone picture, not code	Click **Code** at the top right of the editor.
Indentation lost after pasting	Press Ctrl + Alt + L (Mac: Cmd + Option + L) to auto-format. The code runs either way.
Emulator is very slow	Wait. The first boot can take 1–2 minutes.